

ENGINEERING & PRODUCT CORE

EchoFlow AI

Detailed Functional PRD, Distributed Streaming TRD, & Minimalist Orange UI Design System

Author	Bablu Kumar Singh
Role	Lead Full-Stack Architect
Date	May 2026
Target Architecture	Expo Mobile (iOS/Android) & Hybrid FastAPI/Node.js Framework
Version	1.0.0-MVP Release Specifications

1. Product Requirement Document (PRD)

1.1 Executive Product Statement

EchoFlow AI is an ultra-low-latency, voice-first English language learning platform designed explicitly to build speaking confidence and fluency. Moving away from standard text-based exercises or asynchronous, post-session reports, EchoFlow AI establishes a real-time conversational ecosystem matching the physical sensation of a traditional phone call.

By utilizing a **Bring Your Own Key (BYOK)** financial framework, the solution offers unrestricted scalability to power users without compounding central operational or infrastructure API toll costs.

1.2 Functional Feature Architecture Matrix

1.2.1 Unified BYOK Gateway & Hardware Wallet Integration

The onboarding system requires the configuration of localized API provider validation checks before unlocking runtime features.

- **On-Device Encryption:** The application does not relay or track user API credentials across external web services. Keys are encrypted directly into hardware security modules.
- **Multi-Provider Validation:** Supports real-time runtime routing fallback checks for Gemini Multimodal Live, OpenAI Realtime, Anthropic Claude, and DeepSeek backends.

1.2.2 Context Pre-Flight Engine & Conversational Injectors

Before entering the voice processing pipeline, the application establishes context via three distinct paths:

Initiation Paradigm	Functional System Behavior
Curated Situational Context	Pre-configured contextual vectors tailored to corporate engineering standards (e.g., Agile Scrum standups, stakeholder project signoffs, cross-border architectural syncs).
Custom Text Sandbox	User defined prompts translated into system directives to simulate highly specific environments (e.g., "Practice confronting a landlord about an unauthorized fee").
Serendipity / Random Engine	The execution pipeline defaults context decisions to the LLM backend, forcing the generative AI model to initiate an organic conversation topic dynamically.

1.2.3 Phone Call UI Space & Voice Activity Detection (VAD)

The conversation workspace replicates a classic network phone interface. Running text scripts are explicitly banned during active calls to enhance auditory reliance. The microphone configuration stays persistently

active ("Hot Mic"), relying on advanced silence tracking thresholds to automatically package and transmit audio packets without manual touch interventions.

1.2.4 Real-Time Correction System (Dual-Track Overlay)

Corrections are transmitted asynchronously across independent visual subchannels to prevent conversational interruptions.

The Real-Time Pacing Constraint

To ensure optimal user pacing and emotional safety, visual corrections are throttled via an adaptive windowing formula:

$$\Delta t_{\text{toast}} \geq 30 \text{ ext{ seconds}}$$

Minor grammatical omissions are ignored in real time, prioritising systemic semantic errors (such as past/present tense misuse).

2. Technical Requirement Document (TRD)

2.1 System Architecture Topology

The platform uses a hybrid, low-overhead routing structure to separate raw audio processing streams from administrative setup logic.

- **FastAPI Streaming Cluster:** An asynchronous Python framework processing ultra-low-latency bidirectional WebSockets. It acts as a zero-buffer proxy, connecting mobile devices directly to the Gemini Multimodal Live or OpenAI Realtime API protocols.
- **Node.js Express Proxy:** Coordinates configuration validation, user tracking profiles, and lightweight transactional routing operations.

2.2 Client Native Hardware Pipeline (Expo)

The mobile application utilizes `react-native-live-audio-stream` to pull raw linear audio buffers from the physical mic array, processing data without using the local file system.

```
import LiveAudioStream from 'react-native-live-audio-stream';

// Optimal native configuration for real-time AI audio processing pipelines
const nativeAudioSettings = {
  sampleRate: 16000,      // 16kHz standard sampling frequency
  channels: 1,            // Mono stream allocation to cut bandwidth load in half
  bitsPerSample: 16,      // 16-bit PCM Signed Integer Depth format
  bufferSize: 4096        // Dispatches raw block chunks approximately every 250ms
};

export const initializeMicrophoneStream = (onChunkReceived: (base64Data: string) => void) => {
  LiveAudioStream.init(nativeAudioSettings);
  LiveAudioStream.on('data', (data: string) => {
    // raw base64 encoded audio buffer chunk
    onChunkReceived(data);
  });
};
```

2.3 State Management & Data Storage Architecture

The client app uses a highly optimized state layer and local database cache to prevent re-renders on low-spec hardware during heavy streaming loads:

- **Zustand Micro-Store:** Isolates the active streaming states (e.g., bitrates, transmission status, UI wave shapes) away from React's core rendering loop.
- **Expo SecureStore:** Encrypts and locks secret BYOK provider hashes at the device operating system hardware layer.
- **Local SQLite Cache:** Uses `expo-sqlite` to store conversation metrics and correction records locally.

2.4 Local Database Schema Specification

```
-- Local relational cache for tracking conversational progress analytics
CREATE TABLE IF NOT EXISTS conversation_sessions (
  session_id TEXT PRIMARY KEY NOT NULL,
  topic_title TEXT DEFAULT 'Random Conversation',
  started_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  duration_seconds INTEGER DEFAULT 0
);

CREATE TABLE IF NOT EXISTS real_time_corrections (
  correction_id TEXT PRIMARY KEY NOT NULL,
  session_id TEXT NOT NULL,
  incorrect_phrase TEXT NOT NULL,
  corrected_phrase TEXT NOT NULL,
  grammatical_rule TEXT,
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY(session_id) REFERENCES conversation_sessions(session_id)
);
```

3. Minimalist Orange UI Design System Specification

3.1 Visual Philosophy

The user interface uses a high-contrast dark mode palette to limit eye strain and minimize battery consumption during continuous voice-streaming interactions. It features a modern, premium **Sunset Orange** colorway to focus attention and highlight live connection statuses.

3.2 Style Token Specifications

Token Path	Hex Value	Design Purpose / Visual Intent
colors.bg.primary	#0A0A0B	Primary phone screen canvas color (Pure void black/grey).
colors.bg.secondary	#141416	Surface components, action button background modules, grids.
colors.brand.accent	#F97316	Sunset Orange brand accent for action items and wave visualizers.
colors.feedback.toast-bg	#1C1410	Deep warm amber/orange background tone for correction cards.
colors.feedback.border	#7C2D12	Low-contrast terracotta accent border separating real-time toasts.
colors.text.primary	#FAFAFA	High-readability, crisp white text typography allocation.

3.3 Production Component Schemas

The layout details for the real-time correction component are implemented via standard React Native cross-platform style structures:

```

import React from 'react';
import { StyleSheet } from 'react-native';

export const UIStyleTokens = StyleSheet.create({
  toastOverlayFrame: {
    position: 'absolute',
    bottom: 50,
    left: 20,
    right: 20,
    backgroundColor: '#1C1410',
    borderColor: '#7C2D12',
    borderWidth: 1,
    borderRadius: 16,
    padding: 16,
    flexDirection: 'row',
    alignItems: 'center',
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 4 },
    shadowOpacity: 0.4,
    shadowRadius: 6,
    elevation: 8
  },
  accentBadge: {
    backgroundColor: 'rgba(249, 115, 22, 0.1)',
    borderRadius: 8,
    padding: 8,
    marginRight: 12
  }
});

```

3.4 Production Verification Protocols

Before shipping code to production pipelines, verify the following configuration checklists:

1. **Latency Audits:** Ensure raw PCM streaming roundtrips stay below **1100 ext{ms}** under standard cellular data connections.
2. **Security Sanity Check:** Confirm that the local SQLite database files and raw user key stores are never included inside typical iCloud or Android automatic OS backup files.
3. **VAD Sizing:** Verify the native voice activity thresholds filter out typical environmental noises without clipping the trailing words of native speaker sentences.